

Clase #05 de 27

Strings, Arreglos & Argumentos de Línea de Comando

Mayo 5, Martes

Agenda para esta clase

- Strings en C
- Operaciones de Strings
- Arreglos de Strings & Argumentos de Línea de Comando
- Enunciado Trabajo #1

Strings en C

Strings en C (1 de 3)

```
1 #include<assert.h> //assert
2 #include<string.h> //strlen strcmp
3
4 /* array de chars con \0 como terminador
5  · · +--+--+--+--+--+--+--+--+
6  · · |B·|a·|t·|m·|a·|n·|\0|
7  · · +--+--+--+--+--+--+--+--+
8  */
9
10 int main(){
11     ▶ assert(·····strlen("Batman")·==·6·);
12     ▶ assert(·····sizeof "Batman"·==·7·);
13     ▶ assert(·strcmp("Batman", "Batman"·)·==·0·);
14     ▶ assert(·strcmp("Batman", "Superman")·<·0·);
15     ▶ assert(·strcmp("Batman", "Aquaman"·)·>·0·);
```

Strings en C (2 de 3)

```
17 ▶ char s1[6+1] = {'B', 'a', 't', 'm', 'a', 'n', '\0'};
18 ▶ char s2[] = {'B', 'a', 't', 'm', 'a', 'n', '\0'};
19 ▶ assert( sizeof s1 == sizeof s2 );
20 ▶ char s3[6+1] = {'B', 'a', 't', 'm', 'a', 'n' };
21 ▶ char s4[8+1] = {'B', 'a', 't', 'm', 'a', 'n' };
22 ▶ assert( strcmp("Batman", s1) == 0 );
23 ▶ assert( strcmp("Batman", s2) == 0 );
24 ▶ assert( strcmp("Batman", s3) == 0 );
25 ▶ assert( strcmp("Batman", s4) == 0 );
```


Srtings en C (3 de 3)

```
27 ▶ char s5[6+1] = "Batman";  
28 ▶ char s6[] = "Batman";  
29 ▶ assert(sizeof s5 == sizeof s6);  
30 ▶ char s7[8+1] = "Batman";  
31 ▶ assert(sizeof s7 == 9);  
32 ▶ assert(strcmp("Batman", s5) == 0);  
33 ▶ assert(strcmp("Batman", s6) == 0);  
34 ▶ assert(strcmp("Batman", s7) == 0);  
35 ▶  
36 ▶ char s[] = "Batman";  
37 ▶ assert(s[2] == 't');  
38 ▶ assert("Batman"[2] == 't');
```

Operaciones de Strings

Operación Longitud de una cadena

- Especificación
 - $\text{Length} : \Sigma^* \rightarrow \mathbb{N} / \dots$
 - Recursividad
 - Caso base
 - Paso general
- Implementación
 - Prueba
 - assert
 - Pública
 - Prototipo
 - `size_t Length(const char *s);`
 - Privada
 - Método
 - Recursivo
 - Iterativo
- Versión de la Biblioteca estándar
 - `size_t strlen(const char *s);`

Operación IsEmpty (*EsVacía*)

- Especificación
 - $\text{IsEmpty} : \Sigma^* \rightarrow B$ / $\text{IsEmpty}(s) =$
- Implementación
 - Prueba
 - assert
 - Pública
 - Prototipo
 - `bool IsEmpty(
 const char *s
);`
 - Privada
 - Método
 - Inline
 - Define
- Versión de la biblioteca estándar
 - No tiene.

Operación Concatenación

- Yuxtaposición
 - puede ser de caracteres o de cadenas, pero se puede generalizar como concatenación de cadenas
- Par ordenado, especializado
- Ejemplo
 - $s = ba$
 - $t = bba$
 - $s \cdot t = st = babba$
- ¿Elemento neutro?
- ¿Asociativa?
- ¿Conmutativa?
- Especificación
 - $\text{Concatenar} : \Sigma^* \times \Sigma^* \rightarrow \Sigma^* / \text{Concatenar}(s, t) = s \cdot t = st$
- Implementación
 - ¿Cuántos parámetros?
 - ¿Valor-L Modificable?
 - Prototipo
 - Tres parámetros
 - Dos parámetros
 - Prueba
 - Declaración de variables y resultados
 - Invocación
 - Salida
 - Método
- Biblioteca estándar
 - Prototipo no sea void
 - Interfaz fluida
 - ```
char *strcat(
 char *s,
 const char *t
);
```

# Operación Repetir (Potencia)

## Especificación

- Repetición o Potenciación de una Cadena
  - $ab^3 \neq (ab)^3$
  - $\varepsilon^n = \varepsilon, s^0 = \varepsilon$
- Especificación
  - Repetir :  $\Sigma^* \times \mathbb{N} \rightarrow \Sigma^*$  /  
Repetir (s, n) = especificación Recursiva
  - Pruebas
    - $ababab = \text{Potenciar}(ab, 3)$
    - $\varepsilon = \text{Potenciar}(ababab, 0)$
    - $\varepsilon = \text{Potenciar}(\varepsilon, 3)$
- Implementación
  - Trade-offs similares a Concatenación
- Versión de la biblioteca estándar
  - No tiene.

# Operación Repetir (Potenciar)

## Implementación – Estilos de Interfaz

### Implementación con almacenamiento tomado del *heap* por la función llamada

- `char *Potenciar(  
    const char *,  
    size_t);`
  - Reserva función llamada, libera llamante
- Malloc
  - Análisis de Pre y Postcondiciones
  - `size_t`
  - `sizeof(char)`
  - Heap
  - `void *`
  - Conversiones de tipo
  - Diferencias con C++
    - Casteo necesario
    - Operadores del lenguaje: `new` y `delete`
- Free
  - Análisis de Pre y Postcondiciones

### Implementación con almacenamiento provisto por la función llamante

- `char *Potenciar(  
    const char *,  
    size_t,  
    char *);`
  - Reserva y libera la función llamante
- Mantiene interfaz fluida
- Precondiciones
  - Suficiente espacio

# Operación Repetir (Potenciar)

## Implementación: Pruebas Estilos de Interfaz

```
// Estilo #1
//// ¿Funciona?
assert(0==strcmp("ababab",Potenciar("ab",3)));
assert(0==strcmp("",Potenciar("ababab",0)));
assert(0==strcmp("",Potenciar("",3)));
//// Prototipo
char *Potenciar(const char *, size_t); // Reserva llamada, libera
llamante
//// Produce memory leaks.
```

- Establecer las pre y pos condiciones de cada estilo

```
//// ¿Corrección?
char *s; // result

if(NULL==(s=Potenciar("ab",3))exit(EXIT_FAILURE);
assert(0==strcmp("ababab",s);
free(s);

if(NULL==(s=Potenciar("ababab",0))exit(EXIT_FAILURE);
assert(0==strcmp("",s);
free(s);

if(NULL==(s=Potenciar("",3))exit(EXIT_FAILURE);
assert(0==strcmp("",s);
free(s);
//// ¿Es fácil y cómodo de usar?
```

```
// Estilo #2
char s[6+1]="ab"; //inout
assert(0==strcmp("ababab",Potenciar(s,3));
assert(0==strcmp("",Potenciar(s,0));
assert(0==strcmp("",Potenciar(s,3));
//// Prototipo
char *Potenciar(char *, size_t); // Reserva y libera llamante
//// Más cómodo, aplica interfaz fluida, pero no se pueden pasar literales.
```

```
// Estilo #3
char s[6+1]; //out
assert(0==strcmp("ababab",Potenciar("ab",3,s)));
assert(0==strcmp("",Potenciar("ababab",0,s)));
assert(0==strcmp("",Potenciar("",3,s)));
//// Prototipo
char *Potenciar(const char *, size_t, char *); // Reserva y libera llamante
//// Más cómodo, pero más parámetros.
```

# Operación Repetir (Potenciar)

## Implementación: Dos Métodos

- Aritmética de punteros

- p+len
- string.h
  - strlen
  - strcpy

- **Iterativo**

- for(auto p=t; n--; p+=len)
- strcpy(p,s);

- **Recursivo**

- return
- Potenciar(
  - s,
  - n-1,
  - strcpy(t,s)+len
  - )-len;



# Arreglos de Strings & Argumentos de Línea de Comandos

# Arreglos “Jagged” versus Arreglos Rectangulares

## Punteros versus Arreglos Multidimensionales

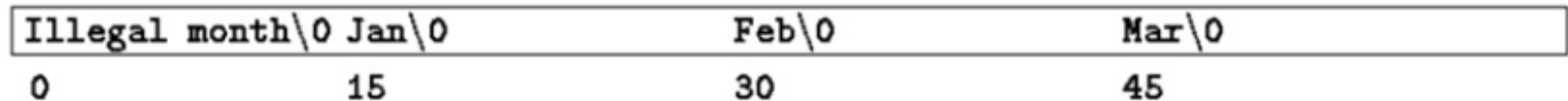
```
int a[10][20]; a[3][4]
```

```
int *b[10]; b[3][4]
```

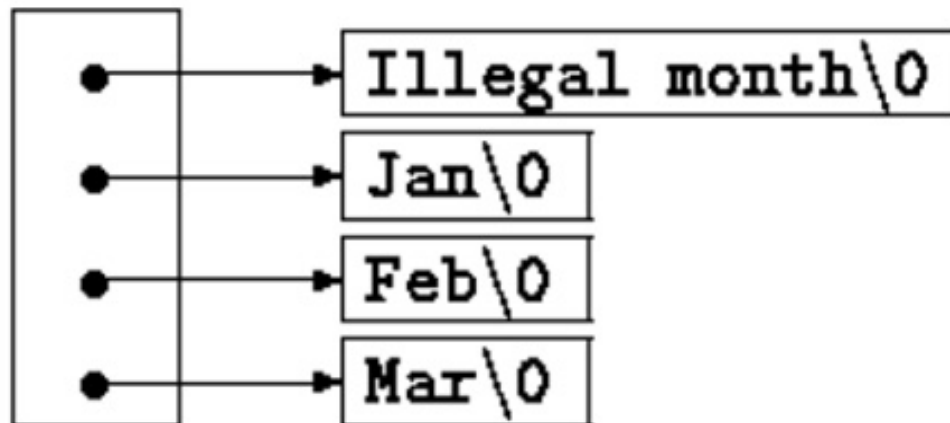
```
char aname[][15]={"Illegal month", "Jan", "Feb", "Mar"};
```

```
char *name[] ={"Illegal month", "Jan", "Feb", "Mar"};
```

**aname:**



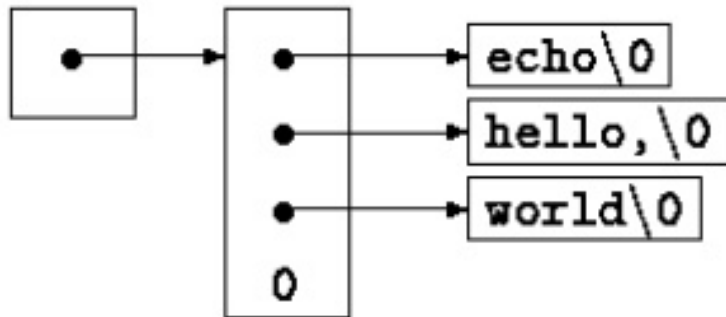
**name:**



# Argumentos de Línea de Comando

```
int main(int argc, char *argv[]){
 for (int i = 1; i < argc; i++)
 printf("%s%s", argv[i],
 (i < argc-1) ? " " : "");
 printf("\n");
}
```

argv:



```
int main(int argc, char *argv[]){
 while (--argc > 0)
 printf("%s%s", *++argv,
 (argc > 1) ? " ": "");
 printf("\n");
}
```

# Preguntas de Repaso

1. ¿Cómo se representan los strings en memoria?
2. ¿En que difieren strlen y sizeof?
3. ¿Qué criterios pueden usarse para la reserva de memoria para strings?
4. ¿Qué es la aritmética de punteros?
5. ¿De qué maneras se puede almacenar en memoria una secuencia de strings?
6. ¿Para qué sirven los argumentos de línea de comando?
7. ¿Cuál es el tipo de dato de argv?

# Términos de la clase

Definir cada término con la bibliografía

- Strings en C
  - Representación de cadenas den memoria
  - strlen
  - sizeof
  - strcmp
  - Declaración e inicialización de arreglos para almacenar cadenas
- Operaciones de Strings
  - Longitud
  - IsEmpty
  - Concatenación
  - strcat
  - Reptetir (Potencia)
- Estilo de interfaz y reserva de almacenamiento
- Aritmética de punteros
- strcpy
- Arreglos de Strings & Argumentos de Línea de Comando
  - Arreglos “Jagged”
  - Arreglos Rectangulares
  - Arreglo de punteros
  - Arreglos multidimensionales
  - Argumentos de Línea de Comando
  - argc
  - argv

# Referencias Bibliográficas

- K&R
  - 5.4 Address Arithmetic
  - 5.10 Command-line Arguments

# Tareas para la próxima clase

1. Comenzar con el trabajo #1



# ¿Consultas?

**Fin de la clase**